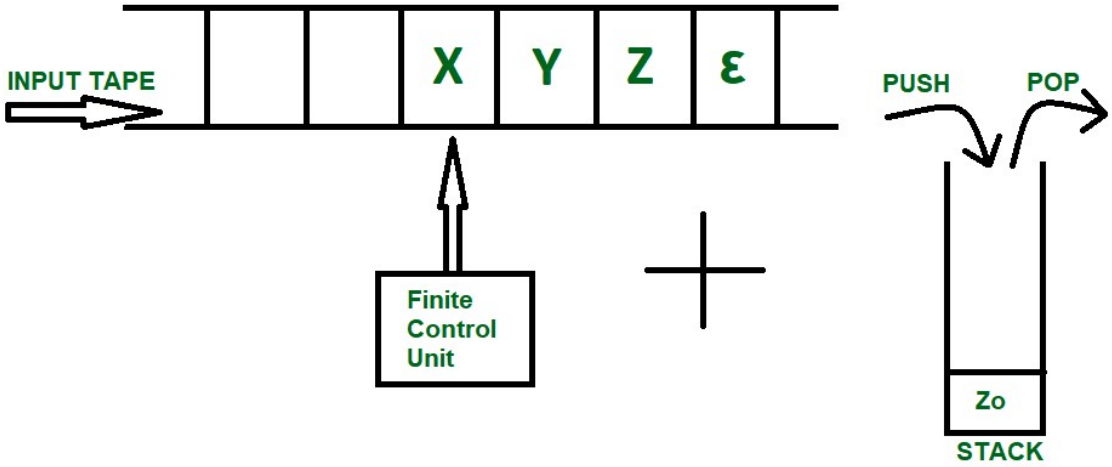
UNIT–IV

Push Down Automata (PDA): Description and definition, Instantaneous Description, Language of PDA, Acceptance by Final state, Acceptance by empty stack, Deterministic PDA, Equivalence of PDA and CFG, CFG to PDA and PDA to CFG.

--by Manish D@nge

Introduction of Pushdown Automata

We have already discussed finite automata. But finite automata can be used to accept only regular languages. Pushdown Automata is a finite automata with extra memory called stack which helps Pushdown automata to recognize Context Free Languages. This article describes pushdown automata in detail.



The diagram above shows a input tape which is how a Finite Automata works, the strings are accepted into the tape and the read header keeps getting updated according the instructions provided by Finite Control Unit. Pushdown Automata on the other hand is a combination of this tape and a Stack data structure. The tuples included to form a PDA are as follows:

PDA = { Q,Σ,qo,F,Zo,Γ,δ }

Q -> Finite set of states
Σ -> Input Alphabets
qo -> Initial State
F -> set of Final States
Zo -> Bottom/Initial Stack Symbol
Γ -> Stack Alphabet
δ -> Transition Function

Pushdown Automata

A Pushdown Automata (PDA) can be defined as :

- Q is the set of states
- ∑is the set of input symbols
- Γ is the set of pushdown symbols (which can be pushed and popped from the stack)
- q0 is the initial state
- Z is the initial pushdown symbol (which is initially present in the stack)
- F is the set of final states
- δ is a transition function that maps Q x {Σ ∪ ∈} x Γ into Q x Γ*. In a given state, the PDA will read the input symbol and stack symbol (top of the stack) move to a new state, and change the symbol of the stack.

Instantaneous Description (ID)

Instantaneous Description (ID) is an informal notation of how a PDA "computes" an input string and makes a decision whether that string is accepted or rejected.

An ID is a triple (q, w, α), where:
1. q is the current state.
2. w is the remaining input.
3.α is the stack contents, top at the left.

Turnstile Notation

⊢ sign is called a "turnstile notation" and represents
one move.
⊢* sign represents a sequence of moves.
Eg- (p, b, T) ⊢ (q, w, α)
This implies that while taking a transition from state p to state q, the input symbol 'b' is consumed, and the top of the stack 'T' is replaced by a new string 'α'

Other Definition :

Understanding Pushdown Automata (PDA):

A **Pushdown Automaton (PDA)** is an advanced computational model that extends finite automata by incorporating a stack as an additional memory structure. This allows PDAs to recognize **context-free languages**, which cannot be processed by finite automata alone. The stack enables the PDA to store and retrieve symbols dynamically, making it suitable for handling nested structures like parentheses or palindromes.

Components of a PDA

A PDA is formally defined as a 7-tuple:

1. **Q**: A finite set of states.
2. **Σ**: The input alphabet.
3. **Γ**: The stack alphabet (symbols that can be pushed or popped).
4. **q_0**: The initial state ($q_0 \in Q$).
5. **Z**: The initial stack symbol ($Z \in \Gamma$).
6. **F**: A set of accepting states ($F \subseteq Q$).
7. **δ**: The transition function, which maps (current state, input symbol, stack top) to (new state, stack operation).

How a PDA Works

The PDA processes input strings by reading symbols, transitioning between states, and manipulating the stack. At each step:

- It reads an input symbol (or ε for no input).
- It checks the top of the stack.
- Based on the transition function, it may push a symbol onto the stack, pop the top symbol, or leave the stack unchanged.

Example: Language $\{a^n b^n \mid n > 0\}$

This language consists of strings with equal numbers of as followed by bs. A PDA for this language operates as follows:

1. Push an A onto the stack for each a encountered.
2. Pop an A from the stack for each b encountered.
3. Accept the string if the stack is empty after processing all input.

Transitions:

$\delta(q_0, a, Z) = (q_0, AZ)$

$\delta(q_0, a, A) = (q_0, AA)$

$\delta(q_0, b, A) = (q_1, \varepsilon)$

$\delta(q_1, b, A) = (q_1, \varepsilon)$

$\delta(q_1, \varepsilon, Z) = (q_1, \varepsilon)$

Instantaneous Description (ID)

An ID represents the current state of the PDA, the remaining input, and the stack contents. It is written as (q, w, α), where:

- q is the current state.
- w is the unprocessed input.
- α is the stack content (top at the left).

The **turnstile notation (⊢)** is used to denote transitions between IDs.

Deterministic vs. Non-Deterministic PDA

- **Deterministic PDA (DPDA)**: At most one transition is possible for a given input and stack symbol.
- **Non-Deterministic PDA (NPDA)**: Multiple transitions may be possible, making it more expressive. Some languages (e.g., $\{a^n b^n c^n\}$) can only be recognized by an NPDA.

Applications of PDA

PDAs are widely used in:

- **Parsing**: Recognizing context-free grammars in compilers.
- **Language Processing**: Validating nested structures like XML or JSON.
- **Automata Theory**: Understanding the hierarchy of computational models.

Understanding PDAs provides insight into how machines process complex patterns and structures, bridging the gap between finite automata and Turing machines.

Pushdown Automata Acceptance by Final State

We have discussed Pushdown Automata (PDA) and its acceptance by empty stack article. Now, in this article, we will discuss how PDA can accept a CFL based on the final state. Given a PDA P as:

P = (Q, Σ, Γ, δ, q0, Z, F)

The language accepted by P is the set of all strings consuming which PDA can move from initial state to final state irrespective of any symbol left on the stack which can be depicted as:

L(P) = {w |(q0, w, Z) =>(qf, ε, s)}

Here, from start state q0 and stack symbol Z, the final state qf ε F is reached when input w is consumed. The stack can contain a string s which is irrelevant as the final state is reached and w will be accepted.

Example: Define the pushdown automata for language {a^nb^n | n > 0} using final state.

Solution: M = where Q = {q0, q1, q2, q3} and ∑ = {a, b} and Γ = { A, Z } and F={q3} and δ is given by:

δ(q0, a, Z) = { (q1, AZ) }

δ(q1, a, A) = { (q1, AA) }

δ(q1, b, A) = { (q2, ε) }

δ(q2, b, A) = { (q2, ε) }

δ(q2, ε, Z) = { (q3, Z) }

Let us see how this automaton works for aaabbb:

Row	State	Input	δ (transition function) used	Stack (Leftmost symbol represents top of stack)	State after move
0	q0	aaabbb		Z	
1	q0	***a***aabbb	δ(q0,a,Z)={(q1,AZ)}	AZ	q1
2	q1	a***a***abbb	δ(q1,a,A)={(q1,AA)}	AAZ	q1
3	q1	aa***a***bbb	δ(q1,a,A)={(q1,AA)}	AAAZ	q1
4	q1	aaa***b***bb	δ(q1,b,A)={(q2, ε)}	AAZ	q2
5	q2	aaab***b***b	δ(q2,b,A)={(q2,ε)}	AZ	q2
6	q2	aaabb***b***	δ(q2,b,A)={(q2,ε)}	Z	q2
7	q2	ε	δ(q2,ε,Z)={(q3,ε)}	Z	q3

Acceptance by Empty Stack vs. Final State in PDA

The statement "If a language is accepted by a PDA through an empty stack, then the same language cannot be accepted by a PDA through a final state" is **false**. Both acceptance methods can be used to recognize the same language, and it is possible to construct a PDA that accepts the same language using either method.

Justification

Acceptance by Empty Stack

In this method, a PDA accepts a string if, after processing the entire input, the stack becomes empty. The final state of the PDA is irrelevant in this case. The language accepted by a PDA through an empty stack is defined as: *[/L(PDA) = {w | (q_0, w, Z_0) ⊢* (q, ε, ε), q ∈ Q}/]* Here, q_0 is the initial state, Z_0 is the initial stack symbol, and ε represents an empty stack.

Acceptance by Final State

In this method, a PDA accepts a string if, after processing the entire input, the PDA reaches a designated final state, regardless of the stack's content. The language accepted by a PDA through a final state is defined as: *[/L(PDA) = {w | (q_0, w, Z_0) ⊢* (qf, ε, s), qf ∈ F}/]* Here, qf is a final state, and s is any stack content.

Equivalence of the Two Methods

It is well-established in automata theory that any language accepted by a PDA through an empty stack can also be accepted by a PDA through a final state, and vice versa. This is because the two acceptance methods are computationally equivalent. A PDA accepting by one method can be converted to accept by the other method without changing the language.

For example:

1. To convert a PDA that accepts by an empty stack to one that accepts by a final state, you can add a new final state and an ε-transition from any configuration where the stack is empty to this new final state.
2. To convert a PDA that accepts by a final state to one that accepts by an empty stack, you can modify the PDA to clear the stack before reaching the final state.

Difference between NPDA and DPDA:

S. No	DPDA(Deterministic Pushdown Automata)	NPDA(Non-deterministic Pushdown Automata)
1.	It is less powerful than NPDA. Example: We can only construct DPDA for odd-length palindromes and not for even length palindromes.	It is more powerful than DPDA. Example: NPDA can be constructed for both even-length and odd-length palindromes.
2.	It is possible to convert every DPDA to a corresponding NPDA.	It is not possible to convert every NPDA to a corresponding DPDA.
3.	The language accepted by DPDA is a subset of the language accepted by NPDA.	The language accepted by NPDA is not a subset of the language accepted by DPDA.
4.	The language accepted by DPDA is called DCFL(Deterministic Context-free Language) which is a subset of NCFL(Non-deterministic Context-free Language) accepted by NPDA.	The language accepted by NPDA is called NCFL(Non-deterministic Context-free Language).
5.	There is only one state transition from one state to	There may or maynot be more than one state transition from one state

S. No	DPDA(Deterministic Pushdown Automata)	NPDA(Non-deterministic Pushdown Automata)
	another state for an input symbol.	to another state for same input symbol.

Deterministic Pushdown Automata

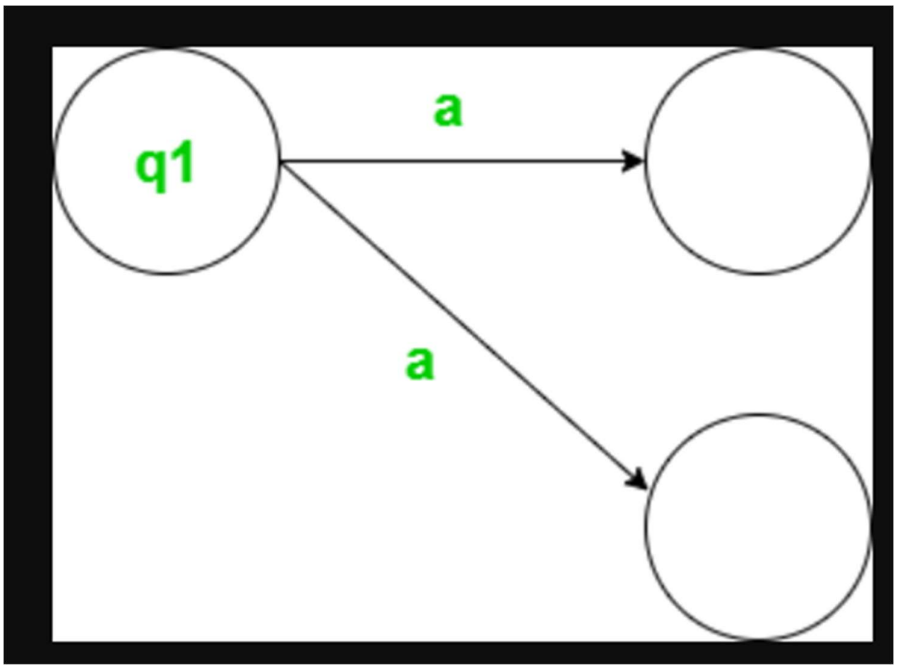
A Pushdown Automata (PDA) can be classified as deterministic if all derivations in the design have to give only a single move. This means that for each combination of state and input symbol, there is only one possible action that the PDA can take.

To understand the difference between deterministic and non-deterministic PDAs, consider the example of finite automata –

Deterministic Finite Automata (DFA)

If we take an input symbol and apply that input symbol on a state, we have to move only to one single move. Single move has to be happened. If we are moving to two different states, that is not deterministic finite automata.

In a DFA, for every combination of state and input symbol, there is only one possible next state. If there are multiple possibilities, the DFA is considered non-deterministic.



Deterministic Pushdown Automata (DPDA)

A DPDA is similar, but it also considers the top symbol on the stack when making a transition. For each combination of state, input symbol, and stack symbol, there is only one possible action.

Properties of Deterministic Pushdown Automata

DPDAs have several important properties that distinguish them from non-deterministic PDAs –

- **Uniqueness of Transitions** – Every transition is unique, meaning there are no duplicate actions for the same combination of state, input symbol, and stack symbol.
- **Predictability** – The behaviour of a DPDA is completely predictable, as there is only one possible action for every situation.
- **Ease of Implementation** – DPDAs are easier to implement than non-deterministic PDAs, as there is no need to handle multiple possible transitions.

Examples of Deterministic Pushdown Automata

Here are two **examples** of languages that can be recognized by DPDAs –

$L = a^n b^{2n}$, $n > 0$: This language consists of strings where the number of 'b's is twice the number of 'a's, and the number of 'a's is greater than 0.

So how to solve this using PDA?

- For every 'a', push it onto the stack.

- For every 'b', pop two 'a's from the stack.
- If the stack is empty at the end of the input, the string is accepted.

L = wcw^R: This language takes any string made up of 0s or 1s (represented by 'w'), adds a 'c' in the middle, followed by the reverse of the first string ('w^R').

So, how to solve this using PDA?

- For every '0' or '1', push it onto the stack.
- When 'c' is encountered, continue pushing symbols onto the stack until the end of the input is reached.
- For every '1' or '0' encountered after 'c', pop the corresponding symbol from the stack.
- If the stack is empty at the end of the input, the string is accepted.

Applications of Deterministic Pushdown Automata

DPDAs have applications in various areas of computer science, including –

- **Formal Language Theory** – DPDAs are used to define and recognize a class of languages known as deterministic context-free languages.
- **Compilers and Interpreters** – DPDAs are used in parsing, the process of converting source code into a form that can be understood by a computer.
- **Natural Language Processing** – DPDAs are used in analyzing and understanding human language.

Conclusion

Deterministic Pushdown Automata are used for contextfree grammars, which are not equivalent to non-deterministic finite automata. It is a powerful tool for recognizing patterns in strings. In this article we have seen the idea of DPDA in detail with definitions and examples with two distinct problems

Equivalence of PDA and CFG:

Pushdown Automata (PDA) and Context-Free Grammars (CFG) are equivalent in their expressive power, meaning that for every PDA, there exists a CFG that generates the same language, and vice versa.

Understanding the Equivalence

1. **Definition**: A **Pushdown Automaton (PDA)** is a type of automaton that uses a stack to manage additional information, allowing it to recognize context-free languages. A **Context-Free Grammar (CFG)** consists of a set of production rules that define how terminal and non-terminal symbols can be combined to generate strings in a language.
2. **Expressive Power**: Both PDAs and CFGs can describe the same class of languages, known as **context-free languages (CFLs)**. This means that any language that can be generated by a CFG can also be recognized by a PDA, and any language recognized by a PDA can be generated by a CFG.

Conversion Between PDA and CFG

- **From CFG to PDA**: To construct a PDA from a given CFG, the PDA simulates the leftmost derivations of the CFG. The stack of the PDA is used to keep track of the non-terminals that need to be expanded. For each production rule in the CFG, the PDA has a corresponding transition that replaces a non-terminal on the stack with the right-hand side of the production.

- **From PDA to CFG**: Conversely, to convert a PDA into an equivalent CFG, we define production rules based on the transitions of the PDA. Each state and stack symbol combination in the PDA corresponds to a non-terminal in the CFG, and the transitions dictate how these non-terminals can be expanded.

Implications of Equivalence

The equivalence between PDAs and CFGs is significant in the field of formal languages and automata theory. It allows for the use of CFGs to specify programming languages while enabling PDAs to implement compilers and interpreters that recognize these langua

ges. This foundational relationship underpins many concepts in computer science, particularly in parsing and language recognition.

Stony Brook University**+1**

In summary, the equivalence of PDAs and CFGs establishes a robust framework for understanding context-
free languages, providing tools for both theoretical exploration and practical application in computer science.

CFG to PDA:

To convert a Context-Free Grammar (CFG) to a Pushdown Automaton (PDA), you can follow a systematic approach that involves simulating the leftmost derivation of the CFG using the PDA's stack.

Steps to Convert CFG to PDA

1. **Define the CFG**: Start with a CFG defined as $G = (N, T, P, S)$, where:

- N is the set of non-terminal symbols.
- T is the set of terminal symbols.
- P is the set of production rules.
- S is the start symbol.

2. **Create a PDA Structure**: Construct a PDA M that simulates the leftmost derivation of the CFG. The PDA will have the following components:

- A single state q (since the PDA will not need to change states during the derivation).
- The stack will initially contain the start symbol S of the CFG.

3. **Transition Rules**: For each production rule in the CFG of the form $A \rightarrow \alpha$ (where A is a non-terminal and α is a string of terminals and non-terminals), add the following transition to the PDA:

- When the top of the stack is A, replace it with α. This can be represented as:

$$(q, A, \gamma) \rightarrow (q, \alpha\gamma)$$

where γ is the rest of the stack content.

4. **Accepting Conditions**: The PDA accepts by empty stack. This means that when the input string is completely read and the stack is empty, the input is accepted as part of the language generated by the CFG.

5. **Example**: For a CFG with the production $S \rightarrow aSb \mid \epsilon$, the PDA would have transitions that allow it to push S onto the stack and pop it while reading a and b from the input, simulating the derivation of strings in the language defined by the CFG.

Converting PDA to CFG

A **Pushdown Automaton (PDA)** is a type of automaton that employs a stack to manage additional information. A **Context-Free Grammar (CFG)** is a set of recursive rewriting rules used to generate patterns of strings. Converting a PDA to an equivalent CFG involves creating a grammar that generates the same language as the PDA.

Steps to Convert PDA to CFG

1. **Identify the Components**: The PDA is defined as (P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)), where (Q) is a finite set of states, (\Sigma) is the input alphabet, (\Gamma) is the stack alphabet, (\delta) is the transition function, (q_0) is the start state, (Z_0) is the initial stack symbol, and (F) is the set of accepting states.

2. **Define Non-Terminals**: For every pair of states ((p, q)) in (Q), create a non-terminal (A_{pq}). This non-terminal represents the PDA transitioning from state (p) to state (q) with an empty stack.

3. **Create Production Rules**: For each transition (\delta(p, a, Z) = (q, \gamma)), where (a) is an input symbol, (Z) is the top stack symbol, and (\gamma) is the string of stack symbols to replace (Z), add the production rule: [A_{pq} \rightarrow a A_{pr_1} A_{r_1r_2} \ldots A_{r_{n-1}q}] where (r_1, r_2, \ldots, r_{n-1}) are intermediate states.

4. **Handle Epsilon Transitions**: For each epsilon transition (\delta(p, \epsilon, Z) = (q, \gamma)), add the production rule: [A_{pq} \rightarrow A_{pr_1} A_{r_1r_2} \ldots A_{r_{n-1}q}]

5. **Add Base Cases**: For each state (p), add the production rule: [A_{pp} \rightarrow \epsilon]